Machine Learning & Deep Learning Mathematics Formula Capsule

Step-by-step mathematical summary of Gradient Descent variants, Backpropagation matrix calculus, Attention mechanisms, Normalization layers, and Loss functions.

LITHAVEN CAMPUS STUDY VAULT • OFFICIAL REVISION CAPSULE

COURSE: CS401 Machine Learning & Neural Networks | **TARGET:** AI/ML Research, Data Science Interviews & GATE AI

1. Loss Functions & Optimization Algorithms

Loss / Optimizer	Mathematical Formulation	Gradient / Update Equation	Primary Application
Mean Squared Error (MSE)	L = (1/N) * sum((y_i - y_hat_i)^2)	dL/dy_hat = -(2/N) * (y - y_hat)	Linear Regression, Continuous Target
Binary Cross-Entropy	L = -(1/N) * sum(y*log(p) + (1-y)*log(1-p))	dL/dz = p - y (when paired with sigmoid)	Binary Classification, Logistic Reg
Categorical Cross-Entropy	L = -sum(y_k * log(p_k))	dL/dz_i = p_i - y_i (with softmax)	Multi-class Classification, Transformers
Adam Optimizer	m_t = beta1*m_{t-1} + (1-beta1)*g_t v_t = beta2*v_{t-1} + (1-beta2)*g_t^2	theta_t = theta_{t-1} - lr * (m_hat / (sqrt(v_hat) + eps))	Default adaptive optimizer for DL models
RMSprop	E[g^2]_t = gamma*E[g^2]_{t-1} + (1-gamma)*g_t^2	theta = theta - (lr / sqrt(E[g^2]_t + eps)) * g_t	Recurrent Neural Networks (RNN/LSTM)

2. Deep Learning Mathematics: Attention & CNN Mechanics

Architecture Component	Mathematical Definition	Dimension Transformation	Intuition & Purpose
Scaled Dot-Product Attention	Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V	Q,K in R^{N x d_k}, V in R^{N x d_v} Output in R^{N x d_v}	Dividing by sqrt(d_k) prevents softmax gradient vanishing at large dimensions.
Multi-Head Attention	MHA = Concat(head_1, ..., head_h) * W^O where head_i = Attention(Q*W_i^Q, K*W_i^K, V*W_i^V)	Projected into h subspaces of size d_model / h	Allows model to jointly attend to information from different representation subspaces.
CNN Output Dimension	O = floor((W - K + 2P) / S) + 1	W: input, K: kernel, P: padding, S: stride	Determines spatial downsampling across conv and max-pooling layers.
Layer Normalization	y = ((x - mu) / sqrt(sigma^2 + eps)) * gamma + beta	Normalized across feature dimension d for each token	Independent of batch size; critical for sequence models and Transformers.

3. Evaluation Metrics & Regularization Diagnostic Matrix

- **Precision & Recall:** Precision = TP / (TP + FP) (Low false positives). Recall = TP / (TP + FN) (Low false negatives / high sensitivity).
- **F1-Score Harmonic Mean:** F1 = 2 * (Precision * Recall) / (Precision + Recall). Use F1 when classes are heavily imbalanced (e.g. fraud detection).
- **L1 vs L2 Regularization:** L1 (Lasso) penalty adds lambda * sum(|w|) causing sparse weights (feature selection). L2 (Ridge) penalty adds lambda * sum(w^2) shrinking weights smoothly.
- **Bias-Variance Tradeoff:** High Bias = Underfitting (add polynomial features, deeper model). High Variance = Overfitting (add Dropout, L2 decay, data augmentation).
- **Vanishing / Exploding Gradients:** Use He initialization for ReLU (std = sqrt(2/n_in)), Xavier/Glorot for Tanh (std = sqrt(2/(n_in + n_out))), and Gradient Clipping at norm threshold.